

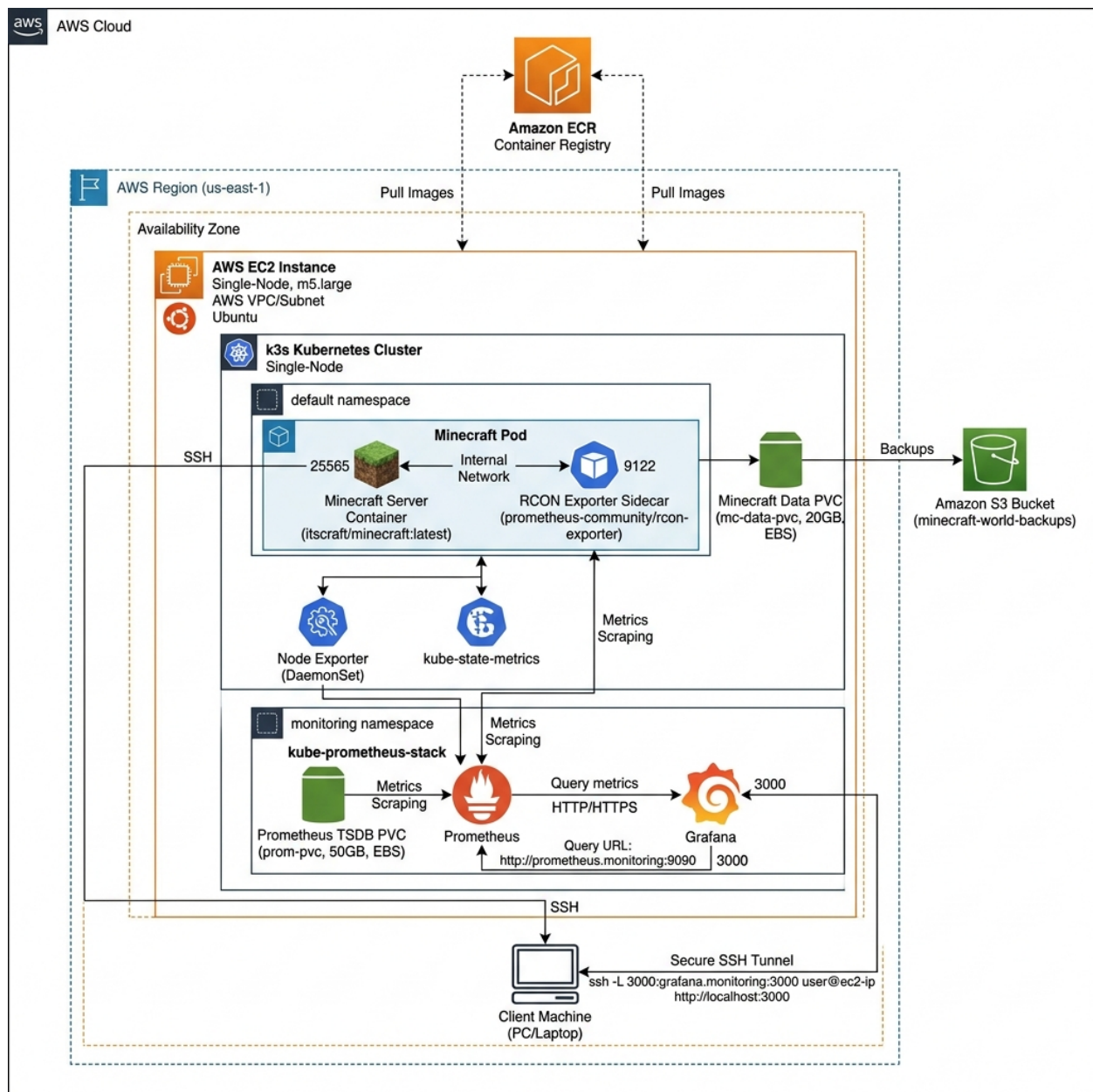
Ops 5: Observability and Incident Response

Minecraft on Kubernetes (k3s) - CS 312 System Administration

Dean Leon | Student ID: 934513949

Repository	https://github.com/GenericProgram/cs312-minecraft-hws
Video	https://media.oregonstate.edu/media/t/1_g1abc3tt
Checkpoints	CP1: 00:03 CP2: 00:38 CP3: 01:10 CP4: 02:07
AWS Region	us-east-1
Instance	t3.large, Ubuntu 22.04 LTS, 20 GB gp3
Kubernetes	k3s v1.35.5+k3s1 (single-node)
Image	itzg/minecraft-server:v1.20.4

1. Architecture



The architecture builds upon the Ops 4 baseline. A kube-prometheus-stack is deployed via Helm into a dedicated monitoring namespace. Prometheus scrapes node health via the Node Exporter daemonset and pod health via kube-state-metrics. Grafana visualizes this data, accessed securely via an SSH port-forward tunnel rather than exposing a public endpoint.

2. Repository File Map

All Ops 5 submission files are organized in the **ops3-4-5/** directory.

helm/monitoring-values.yaml	Declarative configurations, limits, and retention for kube-prometheus-stack.
k8s/monitoring/namespace.yml	Namespace isolation for the observability stack.

k8s/monitoring/prometheus-rules.yml	Declarative Prometheus alerts (CrashLoop, Mem/Disk pressure).
ops-5-writeup/ops_5_writeup.pdf	Self-contained PDF documentation including architecture, runbooks, and incident postmortem.
playbook.yml	Updated Ansible playbook triggering k3s, fixing ECR JSON, and applying secrets.
k8s/configmap.yml	Minecraft environment variables (Pinned to VERSION 1.20.4 to prevent Java 21 crash).
terraform.tfvars	Terraform variables reflecting the t3.large instance scaling.

3. Deployment Runbook

An operator can deploy the full monitoring stack onto the existing Ops 4 baseline by following these steps.

3.1 Prerequisites & Foundation Fixes

Ensure the EC2 instance is scaled to a t3.large via Terraform to accommodate the memory overhead of JVM, Prometheus, and Grafana simultaneously. The Minecraft ConfigMap must explicitly pin VERSION: "1.20.4" to prevent the container from fetching the latest version, which causes a Java 21 initialization crash.

3.2 Install Helm & Deploy Monitoring Stack

SSH into the EC2 instance and run the following to install Helm and the Prometheus stack declaratively:

```
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | sudo bash

sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml helm repo add prometheus-community
https://prometheus-community.github.io/helm-charts
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml helm repo update

sudo k3s kubectl create namespace monitoring

sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml helm install monitoring
prometheus-community/kube-prometheus-stack \
--namespace monitoring \
-f /opt/minecraft-k8s/monitoring-values.yaml
```

3.3 Apply Alerts & Port-Forward Grafana

Apply the Prometheus rules manifest to register the alerts:

```
sudo k3s kubectl apply -f /opt/minecraft-k8s/monitoring/prometheus-rules.yaml
```

To access Grafana without exposing it publicly, establish a secure SSH tunnel from the local WSL machine:

```
ssh -i ~/.ssh/cs312-key.pem -L 3000:localhost:3000 ubuntu@[EC2-IP] \
"sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl port-forward -n monitoring
svc/monitoring-grafana 3000:80"
```

Access the dashboard at `http://localhost:3000`.

4. Observability and Dashboards

The custom **Minecraft Server Health** dashboard aggregates critical metrics to answer the operator question: *"Is my service healthy right now?"* at a single glance.

- **Node Health Panels:** Gauges tracking Node Memory Usage %, Node CPU Usage %, and Node Disk Usage % ensure the underlying EC2 instance is not saturated.
- **Pod Health Panels:** Stats tracking Minecraft Pod Restarts and Pod Ready status. To clean up the visualization and hide historical terminated pods, these queries wrap the raw metric in a `sum()` function.
- **Pod Resource Consumption Panels:** Tracks CPU and memory resource consumption specifically for the Minecraft container (using `container_cpu_usage_seconds_total` and `container_memory_working_set_bytes`) to detect memory leaks and CPU throttling before they affect gameplay.
- **Minecraft-Specific Signal:** The dashboard tracks the `kube_pod_container_status_running` metric specifically for the `minecraft` container. This proves the core application process is alive. While less robust than an application-layer RCON ping, it provides a distinct signal separating the Minecraft container's state from the overall Pod's networking or sidecar states.

5. Alert Runbooks & On-Call Quickstart

On-Call Quickstart: When paged, check the Grafana *Minecraft Server Health* dashboard first. To determine if the outage is user-visible, check the Grafana dashboard's *Pod Ready* and *Service Reachability* panels (or run `nmmap -sV -Pn -p T:25565 [IP]`). If the Minecraft port is closed or the pod ready state is 0, players cannot connect. If the pod is not ready, run `kubectl describe pod` to identify scheduling or image pull errors. Check S3 backup logs if disk usage is full.

5.1 Alert: MinecraftPodCrashLooping

Trigger: `increase(kube_pod_container_status_restarts_total...[10m]) >= 5`

Player-Visible Impact: Yes. Players cannot connect or are actively being disconnected repeatedly.

Threshold Justification: 5 restarts within a 10-minute window indicates a genuine crash loop rather than a standard initialization delay.

First-Response & Resolution Steps:

1. Retrieve the logs of the crashed container's previous execution:

```
sudo k3s kubectl logs -l app=minecraft -c minecraft --previous
```

2. Look for JVM initialization errors. If there is a Java version mismatch, update the ConfigMap to pin the correct version:

```
sudo k3s kubectl patch configmap minecraft-config --type=merge -p '{"data":{"VERSION": "1.20.4"}}'
```

3. Perform a rollout restart to apply the configuration:

```
sudo k3s kubectl rollout restart deployment/minecraft-server
```

5.2 Alert: NodeMemoryPressureHigh

Trigger: Available memory / Total memory * 100 > 85% for 5m.

Player-Visible Impact: Indirectly. May cause server lag or an eventual out-of-memory kill (which occurred to the Grafana pod during setup).

Threshold Justification: 85% provides an early warning buffer, allowing time to scale resources before a hard OOM kill occurs.

First-Response & Resolution Steps:

1. Identify memory-intensive pods on the cluster namespace:

```
sudo k3s kubectl top pods -A
```

2. Identify memory-intensive processes running directly on the host:

```
top -o %MEM -b -n 1 | head -n 20
```

3. If Prometheus or Grafana is consuming excessive memory, update their limits in the monitoring values:

```
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml helm upgrade monitoring  
prometheus-community/kube-prometheus-stack -n monitoring -f  
/opt/minecraft-k8s/monitoring-values.yaml
```

5.3 Alert: NodeDiskUsageHigh

Trigger: Root disk usage is > 80% for 5 minutes.

Player-Visible Impact: Indirectly. World saves and S3 backups will silently fail if disk hits 100%.

Threshold Justification: 80% warns the operator early so they can clear space before world corruption occurs.

First-Response & Resolution Steps:

1. Check host disk usage by directory to locate large files:

```
sudo du -sh /* 2>/dev/null | sort -h
```

2. Prune containerd builder and container image caches:

```
sudo crictl rmi --prune
```

3. Vacuum system journal logs to free space instantly:

```
sudo journalctl --vacuum-time=1d
```

6. Log Investigation & Incident Drill

6.1 Log Investigation

To retrieve raw logs, execute the following command to tail the last 20 lines of the Minecraft server container log:

```
sudo k3s kubectl logs -l app=minecraft -c minecraft --tail=20
```

Look for the line: [Server thread/INFO]: Done (...s)! For help, type "help". This specific event confirms the JVM has finished loading the world and is actively accepting TCP connections.

Log Location & Search Strategy: Logs physically reside on the EC2 host under the containerd runtime directories (specifically at `/var/log/pods/default_minecraft-server-.../*`) and are retrievable via the Kubernetes API. When troubleshooting an incident, the operator should first search for keywords like `WARN`, `ERROR`, `Exception`, or `OutOfMemory`, and verify container initialization by looking for the `Done` startup confirmation.

6.2 Incident Drill: Bad Deployment Rollout

Failure Introduced: An update was deployed utilizing an invalid image tag:

```
sudo k3s kubectl set image deployment/minecraft-server minecraft=...:nonexistent-tag
```

Detection: The `kubectl rollout status` command hung indefinitely. A diagnostic check of the events (`sudo k3s kubectl get events --sort-by=.lastTimestamp`) explicitly highlighted an `ErrImagePull` and subsequent `ImagePullBackOff` error.

Recovery: The deployment was seamlessly reverted to the previous functional `ReplicaSet`:

```
sudo k3s kubectl rollout undo deployment/minecraft-server
```

Confirmation: Executed `nmap -sV -Pn -p T:25565 [IP]` to confirm the port was open and the MOTD broadcasted correctly.

6.3 Incident Postmortem & Prevention

Root Cause: A bad deploy was executed by setting the Minecraft deployment to a non-existent image tag. This caused Kubernetes to transition the pod to an `ErrImagePull` and `ImagePullBackOff` state, halting the rollout.

Recovery: Executed `kubectl rollout undo` to rollback to the previous stable `ReplicaSet`.

Prevention: Implement a pre-push image verification process or a validation step in our deployment scripts to verify that image tags exist in the ECR registry before updating the deployment manifest.

7. Cost Controls & Teardown Checklist

- **Data Retention Limitations:** Prometheus metrics retention is explicitly capped to 3 days (`retention: 3d`) in `monitoring-values.yaml` to prevent EBS disk bloat over time.
- **Memory Guardrails:** Strict memory requests (256Mi) and limits (512Mi) are placed on the Prometheus and Grafana pods to prevent them from starving the Minecraft JVM.
- **Automated Stop Schedule:** The EC2 instance is configured with a nightly stop schedule (e.g. stopped daily at 7:00 PM via AWS Instance Scheduler or cron) to prevent idle compute billing when the lab is not in use.
- **Monitoring Stack Teardown:** Run `helm uninstall monitoring -n monitoring` to instantly drop the resource load of the observability stack.
- **Infrastructure Teardown:** Run `terraform destroy` from `ops3-4-5/` on the local machine after recording completes to terminate the EC2 instance and avoid unwanted billing. Note: This teardown is performed outside the video walkthrough itself.